

Assignment 1: NumPy Array Surgery The Dataset: `np.random.seed(42)`
`data_array = np.random.randint(1, 501, size=(10, 10))` The Question: Using the
`data\_array` provided: 1. Find the global mean of the entire array. Replace all
elements in the array that are greater than this mean with 0\$. 2. Calculate the
sum of each column and the standard deviation of each row . 3. Slice the array
to extract the center 4×4 sub-matrix (rows 3-6, columns 3-6) and
convert it into a flat 1D array.

```
import numpy as np
```

```
# Creating the dataset
```

```
np.random.seed(42)
```

```
data_array = np.random.randint(1, 501, size=(10, 10))
```

```
print("Original Array:\n")
```

```
print(data_array)
```

```
# 1. Find global mean
```

```
global_mean = np.mean(data_array)
```

```
print("\nGlobal Mean:")
```

```
print(global_mean)
```

```
# Replace values greater than mean with 0
```

```
modified_array = np.where(data_array > global_mean, 0, data_array)
```

```
print("\nModified Array:")
```

```
print(modified_array)
```

2. Sum of each column

```
column_sum = np.sum(modified_array, axis=0)
```

```
print("\nColumn Sum:")
```

```
print(column_sum)
```

Standard deviation of each row

```
row_std = np.std(modified_array, axis=1)
```

```
print("\nRow Standard Deviation:")
```

```
print(row_std)
```

3. Extract center 4x4 sub-matrix

```
center_matrix = modified_array[3:7, 3:7]
```

```
print("\nCenter 4x4 Matrix:")
```

```
print(center_matrix)
```

Convert into flat 1D array

```
flat_array = center_matrix.flatten()
```

```
print("\nFlattened 1D Array:")
```

```
print(flat_array)
```

Assignment 2: Pandas Data Cleaning & Transformation

The Dataset: `raw_data = { 'Product': ['Laptop', 'Mouse', 'Monitor', 'Keyboard', 'Laptop', 'Mouse', None], 'Revenue': [1200, 25, 300, 75, 1150, None, 150], 'Cost': [800, 10, 200, 40, 850, 15, 100], 'Date': ['2025-01-10', '2025-02-15', '2025-03-20', '2025-10-05', '2025-11-12', '2025-12-25', '2025-06-01'] }` `df = pd.DataFrame(raw_data)`

The Question: 1. Handle Missing Values: Fill the missing `Revenue` with the median value of the column and the missing `Product` name with "Unknown". 2. Feature Engineering: Create a column `Profit` (`Revenue - Cost`) and a column `Margin\_Percentage` ($(\frac{\text{Profit}}{\text{Revenue}} \times 100)$). 3. Filtering: Convert the `Date` column to datetime objects. Filter the DataFrame to show only products sold in Q4 (October, November, December) that had a `Profit` greater than \$50\$.

```
import pandas as pd
```

```
# Creating the dataset
```

```
raw_data = {  
    'Product': ['Laptop', 'Mouse', 'Monitor', 'Keyboard', 'Laptop', 'Mouse', None],  
    'Revenue': [1200, 25, 300, 75, 1150, None, 150],  
    'Cost': [800, 10, 200, 40, 850, 15, 100],  
    'Date': ['2025-01-10', '2025-02-15', '2025-03-20',  
            '2025-10-05', '2025-11-12', '2025-12-25', '2025-06-01']  
}
```

```
df = pd.DataFrame(raw_data)
```

```
print("Original DataFrame:\n")
```

```
print(df)
```

```
# 1. Handle missing values
```

```
# Fill missing Revenue with median
median_revenue = df['Revenue'].median()
df['Revenue'].fillna(median_revenue, inplace=True)

# Fill missing Product with "Unknown"
df['Product'].fillna("Unknown", inplace=True)

print("\nData After Handling Missing Values:\n")
print(df)
```

2. Feature Engineering

```
# Create Profit column
df['Profit'] = df['Revenue'] - df['Cost']

# Create Margin Percentage column
df['Margin_Percentage'] = (df['Profit'] / df['Revenue']) * 100

print("\nData After Feature Engineering:\n")
print(df)
```

3. Filtering

```
# Convert Date column to datetime
df['Date'] = pd.to_datetime(df['Date'])
```

```
# Filter Q4 data with Profit > 50
q4_products = df[
    (df['Date'].dt.quarter == 4) &
    (df['Profit'] > 50)
]

print("\nQ4 Products With Profit Greater Than 50:\n")
print(q4_products)
```

Assignment 3: Grouping & Pivot Tables The Dataset: employee_data = { 'Dept': ['IT', 'HR', 'IT', 'Sales', 'HR', 'IT', 'Sales', 'Sales'], 'Status': ['FT', 'FT', 'Contract', 'FT', 'Contract', 'FT', 'Contract', 'FT'], 'Salary': [95000, 60000, 70000, 80000, 55000, 98000, 72000, 85000] } emp_df = pd.DataFrame(employee_data) The Question: 1. Group the data by `Dept` and find the average salary and maximum salary for each department. 2. Create a Pivot Table that shows the count of employees, with `Dept` as the index (rows) and `Status` as the columns. 3. Sort the pivot table to identify which department has the highest number of `Contract` workers.

```
import pandas as pd

# Creating the dataset
employee_data = {
    'Dept': ['IT', 'HR', 'IT', 'Sales', 'HR', 'IT', 'Sales', 'Sales'],
    'Status': ['FT', 'FT', 'Contract', 'FT', 'Contract', 'FT', 'Contract', 'FT'],
    'Salary': [95000, 60000, 70000, 80000, 55000, 98000, 72000, 85000]
}
```

```
emp_df = pd.DataFrame(employee_data)
```

```
print("Employee Data:\n")
```

```
print(emp_df)
```

```
# 1. Group by Department and find average and maximum salary
```

```
grouped_data = emp_df.groupby('Dept')['Salary'].agg(['mean', 'max'])
```

```
print("\nAverage and Maximum Salary Department Wise:\n")
```

```
print(grouped_data)
```

```
# 2. Create Pivot Table
```

```
pivot_table = pd.pivot_table(
```

```
    emp_df,
```

```
    index='Dept',
```

```
    columns='Status',
```

```
    values='Salary',
```

```
    aggfunc='count',
```

```
    fill_value=0
```

```
)
```

```
print("\nPivot Table:\n")
```

```
print(pivot_table)
```

```
# 3. Sort Pivot Table by Contract Workers
```

```
sorted_pivot = pivot_table.sort_values(by='Contract', ascending=False)
```

```
print("\nDepartment With Highest Number of Contract Workers:\n")
```

```
print(sorted_pivot)
```